

STACK QNO:8

QUESTION:

Question description

Hassan enjoys jumping from one building to the next. However, he merely jumps to the next higher building and stops when there are none accessible. The amount of stamina necessary for a voyage is equal to the xor of all the heights Hassan leaps till he comes to a halt.

If heights are [1 2 4], and he starts from 1, goes to 2 stamina required is $1 \oplus 2 = 3$, then from 2 to 3. Stamina for the entire journey is $1 \oplus 2 \oplus 4 = 7$. Find the maximum stamina required if can start his journey from any building.

Constraints

$$1 \leq N \leq 10^5$$

$$1 \leq \text{Height} \leq 10^9$$

Input

First line: N , no of buildings.

Second line: N integers, defining heights of buildings.

Output

Single Integer is the maximum stamina required for any journey.

▼ Logical Test Cases

Test Case 1
INPUT (STDIN)
5 1 2 3 8 6
EXPECTED OUTPUT
11

Test Case 2
INPUT (STDIN)
8 1 2 3 8 6 4 7 9
EXPECTED OUTPUT
14

▼ Mandatory Test Cases

Test Case 1
KEYWORD
<code>arr[1000000]</code>

Test Case 2
KEYWORD
<code>for(i=0;i<n;i++)</code>

Test Case 3
KEYWORD
<code>if(arr[i]<arr[j])</code>

Test Case 4
KEYWORD
<code>st[i]=arr[i]^st[j];</code>

▼ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY
7

Test Case 2
TOKEN COUNT
205

Test Case 3
NLOC
26

Source Code:

```
#include <stdio.h>
long long arr [1000000];
long long st[1000000];
int stack[1000000];
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int i;
    for(i=0;i<n;i++) {
        scanf("%lld", &arr[i]);
    }
    int top = -1;
    long long max_stamina = 0;
    for (i = n - 1; i >= 0; i--) {
        while (top != -1 && arr[stack[top]] <= arr[i]) {
            top--;
        }

        if (top == -1) {
            st[i] = arr[i];
        } else {
            int j = stack[top];
            if(arr[i]<arr[j]) {
                st[i]=arr[i]^st[j];
            }
        }

        stack[++top] = i;
        if (st[i] > max_stamina) {
            max_stamina = st[i];
        }
    }
    printf("%lld\n", max_stamina);
    return 0;
}
```

Output:

```
5
1 2 3 8 6
11
```

```
8
1 2 3 8 6 4 7 9
14
```